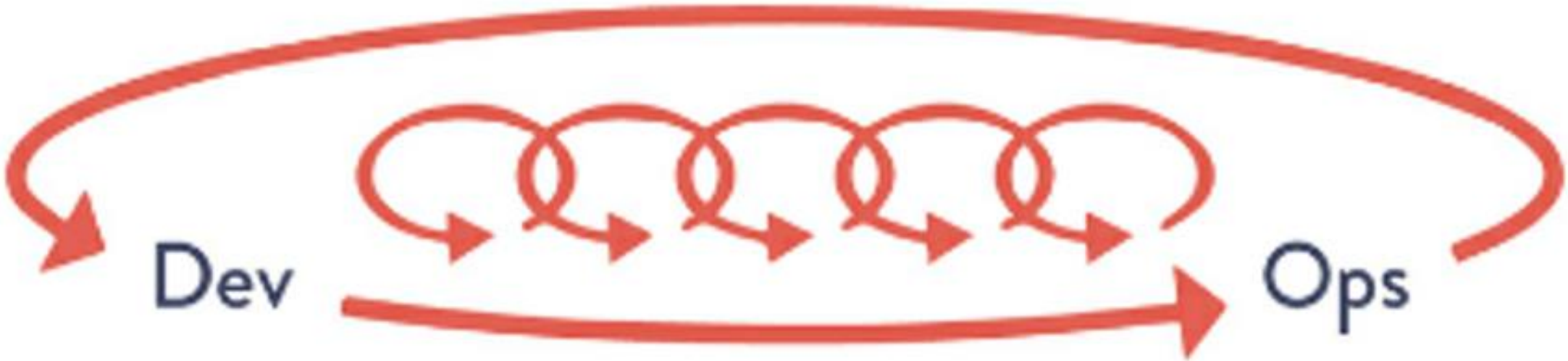
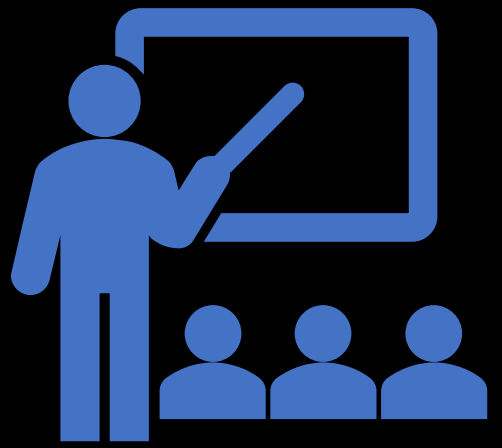
DevOps

3rd Way: Technical Practices of Continual Learning and Experimentation


Dev
Ops

Institutionalize rituals that increase safety, continuous improvement, and learning



Institutionalize rituals that increase safety, continuous improvement, and learning

establish a just culture to make safety possible

inject production failures to create resilience

convert local discoveries into global improvements

reserve time to create organizational improvements and learning

Enable and Inject Learning into Daily Work

To safely work within complex systems

- self-diagnostics
- self-improvement
- be skilled at detecting problems, solving them
- multiplying the effects by making the solutions available throughout the organization.

AWS US-East and Netflix (2011)

- Chaos Monkey

Establish a Just, Learning Culture

Avoid: eliminating error by eliminating the people who caused the errors	“Human error is not our cause of troubles; instead, human error is a consequence of the design of the tools that we gave them” – Dr. Dekker
Just	maximize opportunities for organizational learning, continually reinforcing that we value actions that expose and share more widely the problems in our daily work

Schedule Retrospective Meetings after Accidents Occur

As soon as possible after the accident occurs and before memories and the links between cause and effect fade or circumstances change

Construct a timeline and gather details

Empower all engineers to improve safety by allowing them to give detailed accounts of their contributions to failures

Enable and encourage people who do make mistakes to be the experts who educate the rest of the organization on how not to make them in the future

Schedule Retrospective Meetings after Accidents Occur

Accept that there is always a discretionary space where humans can decide to take action or not, and that the judgment of those decisions lies in hindsight

Propose countermeasures to prevent a similar accident from happening in the future and ensure these countermeasures are recorded with a target date and an owner for follow-up.

Who Should Take Part to the Meeting

- the people involved in decisions that may have contributed to the problem
- the people who identified the problem
- the people who responded to the problem
- the people who diagnosed the problem
- the people who were affected by the problem
- anyone else who is interested in attending the meeting

Example of Countermeasures

- ~~“be more careful” or “be less stupid”~~
- New automated tests to detect dangerous conditions in our deployment pipeline
- Adding further production telemetry
- Identifying categories of changes that require additional peer review
- Conducting rehearsals of this category of failure as part of regularly scheduled game day exercises

Publish Our Retrospective Reviews as Widely as Possible

- translate local learnings and improvements into global learnings and improvements

Decrease Incident Tolerances to Find Ever-Weaker Failure Signals

Decrease Incident Tolerances to Find Ever-Weaker Failure Signals

Amplify weak failure signals is critical to averting catastrophic failures

Instead of treating technology work as entirely standardized, where we strive for process compliance, we must continually seek to find ever-weaker failure signals so that we can better understand and manage the system we operate in

Redefine Failure and Encourage Calculated Risk-Taking

"DevOps must allow this sort of innovation and the resulting risks of people making mistakes. Yes, you'll have more failures in production. But that's a good thing and should not be punished." - Roy Rapoport from Netflix

High performing DevOps organizations will fail and make mistakes more often

Inject Production Failures to Enable Resilience and Learning

Confirm that we have designed and architected our systems properly, so that failures happen in specific and controlled ways

Perform tests to make certain that our systems fail gracefully

First define failure modes and then perform testing to ensure that these failure modes operate as designed

Injecting faults into production environment and rehearsing large-scale failures so we are confident we can recover from accidents when they occur, ideally without even impacting our customers

Institute Game Days to Rehearse Failures

Help teams simulate and rehearse accidents to give them the ability to practice

Schedule a catastrophic event, such as the simulated destruction of an entire data center, to happen at some point in the future

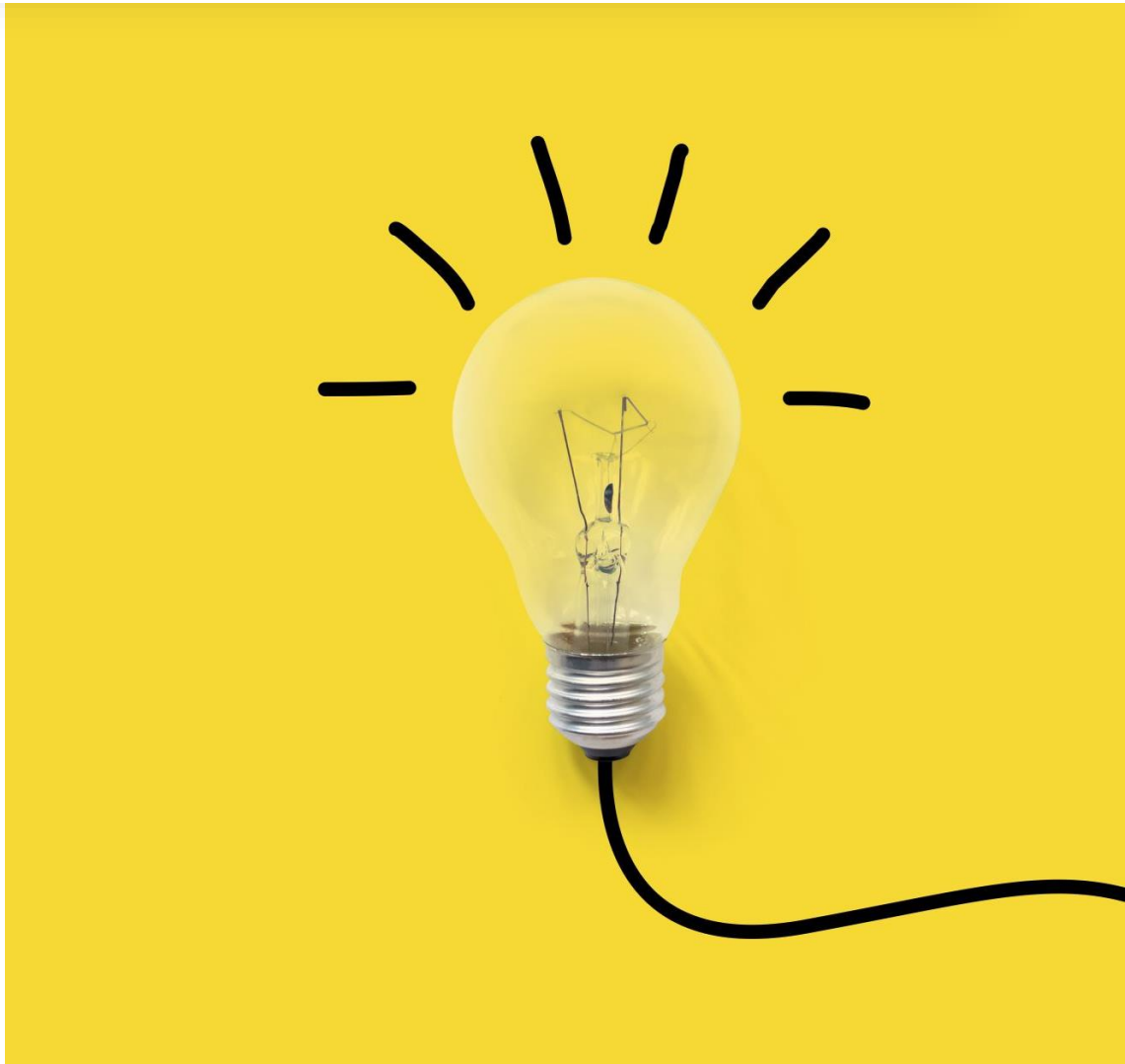
Teams have time to prepare, to eliminate all the single points of failure, and to create the necessary monitoring procedures, failover procedures

At the scheduled time, execute the outage

Convert Local Discovery into Global Improvements

Convert Local Discovery into Global Improvements

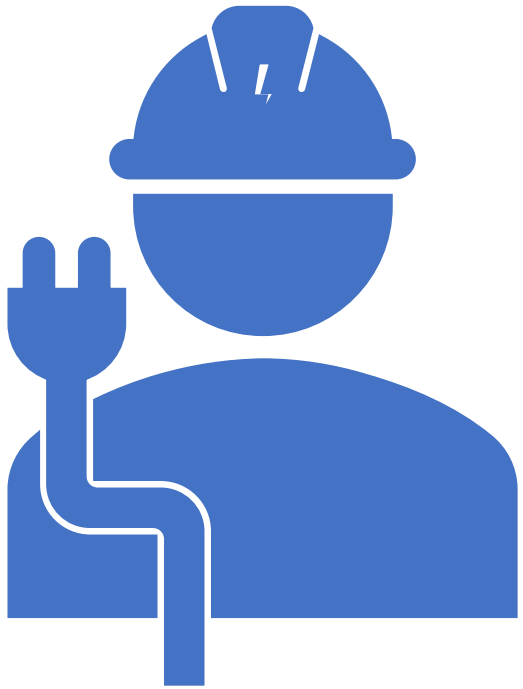
Create mechanisms that make it possible for new learnings and improvements discovered locally to be captured and shared globally throughout the entire organization, multiplying the effect of global knowledge and improvement



Use Chat Rooms and Chat Bots to Automate and Capture Organizational Knowledge

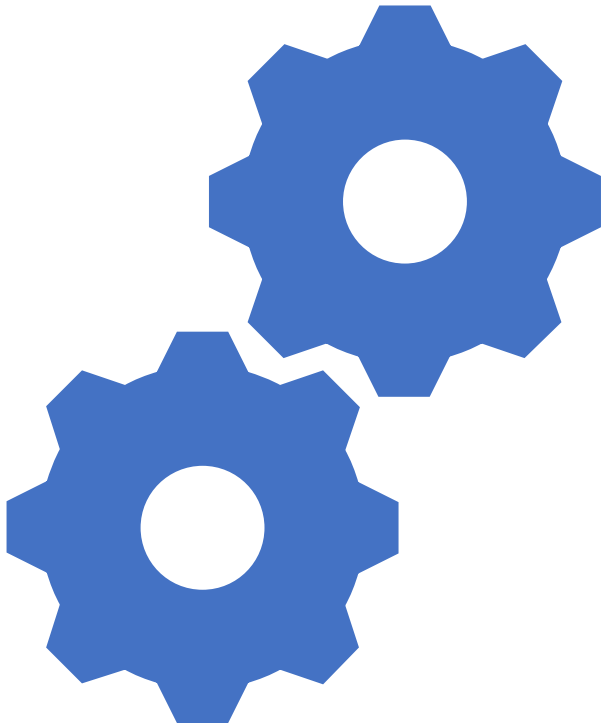
FACILITATE FAST COMMUNICATION WITHIN TEAMS

TRIGGER AUTOMATION



Benefits of using Chat Rooms

- Everyone see everything that is happening
- Engineers, on their first day of work, could see what daily work looked like and how it was performed
- People encouraged to ask for help when they see others helping each other
- Rapid organizational learning is enabled and accumulated



Automate Standardized Processes in Software for Reuse

Automate Standardized Processes in Software for Reuse

- Instead of putting expertise into Word documents, put them into an executable form that makes them easier to reuse
- Use a centralized repository
- Enable the process to be widely adopted, providing value to anyone who uses them

Create a Single, Shared Source Code Repository for the Entire Organization

- Any update in the source code repository (e.g., a shared library), is rapidly and automatically propagated to every other service that uses that library, and it is integrated through each team's deployment pipeline
  - configuration standards for libraries, infrastructure, and environments (Chef, Puppet, or Ansible scripts)
  - deployment tools
  - testing standards and tools, including security
  - deployment pipeline tools
  - monitoring and analysis tools
  - tutorials and standards

DORA's 2019 State of DevOps Report

Teams that manage code maintainability well have systems and tools that make it easy for developers to change code maintained by other teams, find examples in the codebase, reuse other people's code, as well as add, upgrade, and migrate to new versions of dependencies without breaking their code. Having these systems and tools in place not only contributes to CD, but also helps decrease technical debt, which in turn improves productivity

Updating dependencies

Gets very easy with single repository

Alternative organization-wide package repository

For security reasons it is essential to ensure that dependencies are drawn only from within the organization’s source control repository or package repository (“software supply chain”)

Spread Knowledge by Using Automated Tests as Documentation and Communities of Practice

TDD: turns test suites into a living, up-to-date specification of the system

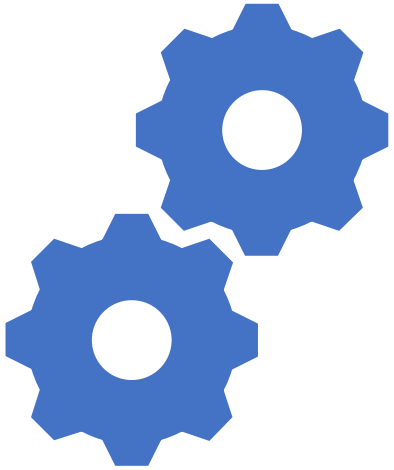
Team responsible of the shared library is also responsible to migrate everybody to latest version, so that we only have one version in production

Quick detection of regression errors through comprehensive automated testing and continuous integration for all systems that use the library

Dedicated communication channel

Design for Operations through Codified Non-Functional Requirements

Design for Operations through Codified Non-Functional Requirements



- Integrate a set of non-functional requirements into all products and services
- Examples of non-functional requirements
  - sufficient production telemetry in our applications and environments
  - the ability to accurately track dependencies
  - services that are resilient and degrade gracefully
  - forward and backward compatibility between versions
  - the ability to archive data to manage the size of the production data set
  - the ability to easily search and understand log messages across services
  - the ability to trace requests from users through multiple services
  - simple, centralized runtime configuration using feature flags

Build Reusable Operations User Stories into Development

When there is Operations work that cannot be fully automated or made self-service, our goal is to make this recurring work as repeatable and deterministic as possible

We should automate as much of this work as possible

Collectively define the handoffs as clearly as possible to reduce lead times and errors

Create well-defined “Ops user stories” that represent work activities that can be reused across all our projects (e.g., deployment, capacity, security, etc.)

Ensure Technology Choices Help Achieve Organizational Goals

Ensure Technology Choices Help Achieve Organizational Goals

When having service-oriented architectures, small service teams can potentially build and run their service in whatever language or framework best serves their specific needs

This can result in a bottleneck: we may have optimized for team productivity but inadvertently impeded the achievement of organizational goals

Operations can influence which components are used in production or give them the ability to not be responsible for unsupported platforms

Identify technologies that

Slow down the flow

Create heigh level of unplanned work

Create large numbers of support requests

Inconsistent with desired architectural outcomes

Improvement Blitz

Comes from Toyota Production system

In Software we want to focus on improving our daily work, e.g., solving daily workarounds

One week in which everyone in the organization works on an improvement activity at the same time

At the end each one present the problem they worked on and the outcomes

Create the organizational culture and norms to encourage everyone to continually find and fix issues

Allocated time periodically for everybody to either learn or teach

Enable Everyone to Teach and Learn

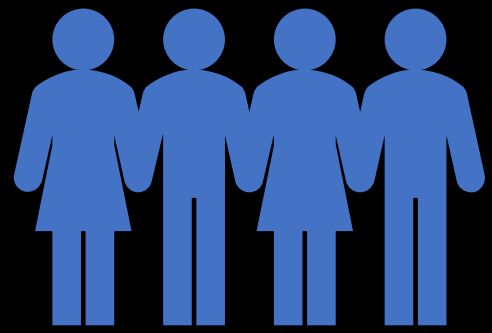
Share your Experiences from DevOps Conferences

Attend conferences

Give talks

Create/Organize internal/external conferences

Create Community Structures to Spread Practices



Thank you